

Getting started: correcting an estimate for linkage error

Contents

The problem	1
What you need: two noisy measures	1
The data	2
Step 1: tabulate	2
Step 2: estimate the misclassification model	3
Step 3: read the output	4
Step 4: turn Π into a regression coefficient	5
Does the correction bite?	6
Where to go next	6

The problem

You have linked historical records: fathers to sons, censuses to censuses, draft cards to death certificates. You link them with a name-matching algorithm, you get a linked sample, and you run a regression on it.

Some of those links are wrong. Not many, perhaps — 10%, 20% — but wrong. And a wrong link does not put a random number in your outcome column; it puts *somebody else's* outcome there. Somebody else drawn, roughly, from the pool of people whose names looked similar.

The consequence for your regression is not subtle. Suppose you regress a son's income on his father's income to estimate intergenerational persistence. When a son is mis-linked, his recorded income is really a stranger's income, which is uncorrelated with *this* father's income. The slope you estimate is a mixture of the true slope (for correctly linked pairs) and zero (for the rest). You get attenuation, and the size of the attenuation is roughly the false-link rate. A 20% link error rate knocks 20% off your estimate of social mobility's opposite.

The usual fixes are unsatisfying. You can tighten the linking algorithm, which trades link errors for selection bias. You can bound the damage under worst-case assumptions, which is honest but wide. Or you can *model* the error, estimate its distribution from the data, and correct the estimate. That is what this package does.

What you need: two noisy measures

The one non-negotiable requirement is that the outcome be measured **twice**.

Here is why, without any econometrics. With a single measure you cannot tell a genuinely poor son from a rich son whose record was mis-linked to a poor stranger. Both appear as “poor” in your data, and nothing in the data separates them. Add a second measure of the same underlying outcome — the son observed again in a later census, say — and the separation becomes possible. When the two measures agree, the value is probably right. When they disagree, at least one is wrong. *How often* they disagree, and *in which direction*, is exactly the information you need to pin down the error rates.

Once you know the error rates, you can undo them: the distribution you actually observe is the true distribution passed through a known “smearing” matrix, and you invert the smearing.

Three conditions make this work.

1. **Two measures.** Call them Y_1 and Y_2 . They must be measures of the same latent outcome Y^* .

2. **Conditionally independent errors.** Given the truth, the two measures must err independently. Two transcriptions by the same clerk from the same page are *not* two measures — they share their mistakes. Two different censuses, or a linked record and an independently collected self-report, usually are.
3. **Errors unrelated to the regressor.** Given the truth, the errors must not depend on X . If the linking algorithm is worse for poor families than rich ones, condition on family status using the `controls` argument so that this holds within each control cell.

There is a fourth condition that is more of a normalization: the modal report is the truth. Most sons are recorded correctly. Without this the model cannot tell which of two candidate solutions is the “true” outcome and which is the error, and the package would happily hand you the relabeled one.

The data

The package ships a synthetic dataset, `ancienregime`: 100,000 father–son pairs in Third Estate France, linked across three censuses (that never happened) in 1750, 1770, and 1780. The son’s occupation and income score are recorded **twice**, in 1770 and again in 1780, which supplies the two measures.

```
data(ancienregime)
str(ancienregime)
#> 'data.frame': 100000 obs. of 9 variables:
#> $ birthplace : chr "Languedoc" "Normandie" "Ile-de-France" "Normandie" ...
#> $ birthyear : num 1740 1750 1750 1740 1745 ...
#> $ linked_weight : num 0.499 1.658 1.83 0.725 3.954 ...
#> $ father_occupation_1750: chr "Petite Bourgeoisie" "Journalier" "Metayer" "Metayer" ...
#> $ father_income_1750 : num 1681 479 232 196 336 ...
#> $ son_occupation_1770 : chr "Petite Bourgeoisie" "Metayer" "Metayer" "Metayer" ...
#> $ son_income_1770 : num 1681 194 232 196 336 ...
#> $ son_occupation_1780 : chr "Petite Bourgeoisie" "Metayer" "Metayer" "Metayer" ...
#> $ son_income_1780 : num 1681 194 232 196 136 ...
```

The two measures of the son disagree for a substantial share of pairs:

```
mean(ancienregime$son_occupation_1770 != ancienregime$son_occupation_1780)
#> [1] 0.31371
```

That disagreement is not a nuisance. It is the identifying variation.

Step 1: tabulate

`misclassifyr` works from a tabulation, not from microdata. The estimator only ever touches the counts of each (X , Y_1 , Y_2) cell, so `prep_misclassification_data()` builds that table and records the bookkeeping the estimator needs — how many categories, what they are called, and what numeric score each category carries.

Occupation is categorical, but we want a regression coefficient in income units. The `_bin` arguments handle exactly this: bin the continuous income variable by occupation, and record the within-bin mean income as the numeric score for each category.

```
inputs <- prep_misclassification_data(
  data = ancienregime,
  outcome_1 = "son_income_1780",
  outcome_2 = "son_income_1770",
  regressor = "father_income_1750",
  outcome_1_bin = "son_occupation_1780",
  outcome_2_bin = "son_occupation_1770",
  regressor_bin = "father_occupation_1750",
  weights = "linked_weight",
```

```

record_vals = TRUE,
round_vals  = 0
)

c(J = inputs$J, K = inputs$K)      # categories of the outcome and the regressor
#> J K
#> 6 6
nrow(inputs$tab)                    # a balanced  $J^2 * K$  table
#> [1] 216
head(inputs$tab)
#>           X           Y1           Y2           n
#> 1  35.18115 36.02765 36.48623 52.6672904
#> 2 203.07200 36.02765 36.48623 675.8423026
#> 3 469.76280 36.02765 36.48623 662.7342262
#> 4 879.69133 36.02765 36.48623 44.5453191
#> 5 1678.53368 36.02765 36.48623 33.8282703
#> 6 4984.25329 36.02765 36.48623 0.8899815
round(inputs$Y_vals)                # mean income within each occupation
#> [1] 36 201 466 881 1680 4946

```

J is the number of outcome categories, K the number of regressor categories, and the tabulation has $J^2 * K = 216$ rows: one for every combination of the father's occupation and the two recorded occupations of the son.

Step 2: estimate the misclassification model

`misclassifyr()` maximizes the likelihood of that tabulation over two objects at once:

- Π , the joint distribution of the regressor X and the **latent** outcome Y^* — the thing you actually want; and
- Δ , the distribution of the two recorded values given the truth, $\Pr(Y_1, Y_2 | Y^*)$ — the smearing matrix.

You choose the shape of Δ through the `model_to_Delta` argument. Here we use `model_to_Delta_RL_ind`, the record-linkage model: with probability $1 - \alpha$ the link is correct and the recorded occupation is the true one; with probability α the link failed and the recorded occupation is an independent draw from a distribution ρ over occupations. That is a natural description of name-matching failure, and it costs only $4J - 2$ parameters rather than the $2J(J - 1)$ of an unrestricted model.

```

set.seed(1)      # the stability check below perturbs starting values
fit <- misclassifyr(
  tab          = inputs$tab,
  J            = inputs$J,
  K            = inputs$K,
  X_names      = inputs$X_names,
  Y1_names     = inputs$Y1_names,
  Y2_names     = inputs$Y2_names,
  X_vals       = inputs$X_vals,
  Y_vals       = inputs$Y_vals,
  model_to_Delta = model_to_Delta_RL_ind,
  mle          = TRUE,
  bayesian     = FALSE,
  makeplots    = FALSE
)
#> Warning in estimate_misclassification(misclassification_inputs): Optimal theta
#> is inconsistent across starting locations.

```

Step 3: read the output

`misclassify()` returns a long list. Four pieces matter most.

The estimated joint distribution. `Pi_hat_mle` is a length- $J \times K$ vector holding the J by K matrix column-major: rows are the latent outcome, columns the regressor.

The categories are labelled by their mean income, since that is what we binned on; they are the six occupations, in ascending order of income.

```
occ <- c("Vagabond", "Metayer", "Journalier",
        "Petit Metiers", "Petite Bourgeoisie", "Haute Bourgeoisie")

Pi_hat <- matrix(fit$Pi_hat_mle, nrow = inputs$J,
                dimnames = list(`Son (latent)` = occ, Father = occ))
round(Pi_hat, 3)
```

#>		Father			
#>	Son (latent)	Vagabond	Metayer	Journalier	Petit Metiers
#>	Vagabond	0.009	0.062	0.060	0.007
#>	Metayer	0.006	0.331	0.098	0.017
#>	Journalier	0.000	0.023	0.156	0.018
#>	Petit Metiers	0.002	0.008	0.029	0.058
#>	Petite Bourgeoisie	0.002	0.010	0.029	0.011
#>	Haute Bourgeoisie	0.000	0.002	0.002	0.002

#>		Father	
#>	Son (latent)	Petite Bourgeoisie	Haute Bourgeoisie
#>	Vagabond	0.005	0.000
#>	Metayer	0.007	0.001
#>	Journalier	0.008	0.000
#>	Petit Metiers	0.006	0.001
#>	Petite Bourgeoisie	0.023	0.000
#>	Haute Bourgeoisie	0.000	0.009

The estimated error rates. `Delta_hat_mle` holds the J by J^2 joint distribution of both measures given the truth. To read off the more intuitive object — $\Pr(Y_1 | Y^*)$, the chance a son whose true occupation is j is recorded as k in 1780 — marginalize over the second measure:

```
marginalize <- do.call(rbind, replicate(inputs$J, diag(inputs$J), simplify = FALSE))
Delta1 <- matrix(fit$Delta_hat_mle, nrow = inputs$J) %*% marginalize
dimnames(Delta1) <- list(Truth = occ, `Recorded 1780` = occ)
round(Delta1, 3)
```

#>		Recorded 1780			
#>	Truth	Vagabond	Metayer	Journalier	Petit Metiers
#>	Vagabond	0.348	0.185	0.262	0.088
#>	Metayer	0.008	0.933	0.033	0.011
#>	Journalier	0.036	0.104	0.745	0.050
#>	Petit Metiers	0.042	0.121	0.171	0.590
#>	Petite Bourgeoisie	0.040	0.117	0.165	0.056
#>	Haute Bourgeoisie	0.016	0.045	0.064	0.021

#>		Recorded 1780	
#>	Truth	Petite Bourgeoisie	Haute Bourgeoisie
#>	Vagabond	0.107	0.009
#>	Metayer	0.014	0.001
#>	Journalier	0.060	0.005
#>	Petit Metiers	0.070	0.006
#>	Petite Bourgeoisie	0.616	0.006

```
#> Haute Bourgeoisie 0.026 0.828
```

Careful with the orientation: **rows are the truth**, so row j is the distribution of what gets *recorded* for a son whose occupation really is j . The diagonal is the probability of being recorded correctly, and one minus the diagonal is the implied error rate:

```
round(1 - diag(Delta1), 3) # implied error rate by true occupation
#> Vagabond Metayer Journalier Petit Metiers
#> 0.652 0.067 0.255 0.410
#> Petite Bourgeoisie Haute Bourgeoisie
#> 0.384 0.172
```

These are worth staring at. They are not uniform — the model says the lowest-status category is recorded correctly far less often than the middling ones, which is what you would expect if vagabonds are hard to find, hard to link, and easy to confuse with day labourers. If instead they had come back implausible (60% error where the linkage literature reports 10%, or a rate of zero) that would be evidence against the shape of Δ you chose, not a result.

Two diagnostics you should always look at.

```
fit$fisher_info_err # was the information matrix invertible?
#> [1] "Fisher information matrix is invertible."

c(elements_of_theta = inputs$J * inputs$K + inputs$J^3,
  inconsistency = fit$inconsistency_mle)
#> elements_of_theta inconsistency
#> 252.0000000 0.6943207
```

The first tells you whether analytical standard errors are trustworthy; here they are.

The second re-runs the optimizer from a perturbed starting value and reports the *total* absolute distance between the two answers, summed over every element of Π and Δ . `misclassify()` warns above 0.1, which is a blunt threshold: it does not scale with the size of the model, and this model has 252 elements. Per element the discrepancy is around 0.003, small enough to be convergence slop rather than a second optimum. Still, a fired warning is a prompt to check whether the number you care about is actually sensitive to it, and `vignette("inference")` shows how to tell — and what to do when it genuinely is.

Step 4: turn Π into a regression coefficient

`Pi_to_beta()` computes the slope of the population least-squares projection of Y^* on X implied by the estimated joint distribution, together with a delta-method standard error.

```
corrected <- Pi_to_beta(
  X_vals = inputs$X_vals,
  Y_vals = inputs$Y_vals,
  mle = TRUE,
  Pi_mle = fit$Pi_hat_mle,
  cov_Pi = fit$cov_Pi_mle
)

c(beta = unname(corrected$beta_hat_mle),
  se = unname(corrected$se_beta_mle))
#> beta se
#> 0.724128896 0.008575661
```

Does the correction bite?

Compare against what you would have reported without it. The cleanest comparison holds the discretization fixed and simply *skips* the correction: take the observed joint distribution of the 1780 measure and the father's occupation and compute the same slope from it.

```
tab <- inputs$tab

naive_from <- function(measure) {
  P <- tapply(tab$n, list(tab[[measure]], tab$X), sum)
  Pi_to_beta_inner(c(P / sum(P)), inputs$X_vals, inputs$Y_vals, 1)
}

results <- data.frame(
  estimate = c("Naive, 1780 measure",
               "Naive, 1770 measure",
               "Individual-level OLS",
               "Misclassification-corrected"),
  beta = c(naive_from("Y1"),
            naive_from("Y2"),
            unname(coef(lm(son_income_1780 ~ father_income_1750,
                           data = ancienregime,
                           weights = ancienregime$linked_weight))[2]),
            unname(corrected$beta_hat_mle))
)

results$ratio_to_corrected <- results$beta / unname(corrected$beta_hat_mle)
knitr::kable(results, digits = 3)
```

estimate	beta	ratio_to_corrected
Naive, 1780 measure	0.586	0.809
Naive, 1770 measure	0.628	0.867
Individual-level OLS	0.595	0.822
Misclassification-corrected	0.724	1.000

The naive estimates come in 13% to 19% below the corrected one, which is the order of magnitude the estimated error rates above imply. Reported as an intergenerational elasticity in a paper, that gap is the difference between a society you would describe as fairly mobile and one you would describe as notably less so — from the same data, with the same linking algorithm.

Note also that the two naive estimates disagree with each other (0.59 versus 0.63). They are estimates of the same quantity from the same sample; the gap between them is measurement error making itself visible. When your results move depending on which of two equivalent measures you happen to use, that is a signal to model the error rather than pick a favourite.

Where to go next

- **vignette("designing-misclassification-models")** — the important one. The record-linkage model used above is one shape of Δ among many, and it will not describe *your* errors. This vignette shows how to enumerate the error channels in your own setting, turn them into a parameterized Δ , write it as an R closure, and verify it. It also covers using a large language model as a drafting assistant, and how to check what it hands you.
- **vignette("sparse-and-large")** — when the outcome has hundreds or thousands of categories (counties, detailed occupation codes) the balanced tabulation above becomes impossible and the general estimator too slow. Sparse tabulations and a dedicated expectation-maximization estimator handle it.

- `vignette("inference")` — delta-method standard errors, what a singular information matrix means, and Monte Carlo confidence sets that remain valid when the model is only partially identified.